

Efficient Resource Management of Kubernetes Pods using Artificial Intelligence

Karanbir Singh

Infrastructure Lifecycle Management
Cloud, Salesforce
San Francisco, USA

<https://orcid.org/0009-0005-2655-6335>

Deepak Muppiri

Infrastructure Lifecycle Management
Cloud, Salesforce
San Francisco, USA

<https://orcid.org/0009-0001-5818-3247>

Inderveer Chana

Computer Science & Engineering
Department

Thapar Institute of Engineering and
Technology, Patiala, India
inderveer@thapar.edu

Abstract— In the realm of cloud infrastructure management, efficient resource utilization is a classic problem. It is imperative to use cloud resources optimally to make them more cost effective. A resource can be a unit of compute, memory, etc. Considerable research and development have been done to efficiently run a cloud service such that it can serve varying requirements of the consumers at the same time. Tools such as Kubernetes are widely used in the industry to auto scale services based on their demand. The identification of adequate sets of resources that are just right for the services at the pod level is still a challenge. This paper presents an efficient resource management model at the Kubernetes pod level with the help of Artificial Intelligence.

Keywords— Resource Management, Artificial Intelligence, Cloud Computing, Autoscaling, Kubernetes, Pods, Deep Learning

I. INTRODUCTION

Cloud computing operates on a resource-sharing model where tenants pay based on their usage. For cloud vendors, energy consumption is a critical consideration, making efficient resource utilization and energy efficiency the key objectives. To achieve effective resource utilization, systems need the ability to allocate and deallocate resources dynamically — a feature known as auto-scaling. Significant research has focused on enhancing auto-scaling capabilities for cloud platforms. Lorido-Botran et al. conducted an extensive review of auto-scaling techniques for cloud, classifying them into reactive, proactive, and predictive approaches as shown in Fig. 1. Their study also underscores the challenges associated with predictive auto-scaling due to the dynamic nature of cloud workloads [1].

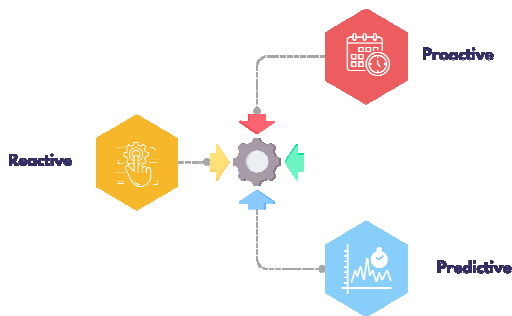


Fig. 1. Types of Autoscaling

Predictive auto-scaling involves forecasting the adequate amount of resources required for an application running in the cloud without compromising Service Level Agreements (SLAs) and Quality of Service (QoS). Also, researchers have devised multiple methods to achieve predictive auto-scaling at the underlying layers of cloud infrastructure, including the Regressive Ensemble Approach for Prediction (REAP) [2].

Kubernetes is a portable, extensible, open-source platform container orchestration platform that facilitates both declarative configuration and automation [3]. Modern applications increasingly rely on Kubernetes (K8s) as it enables developers to implement auto-scaling features easily. The K8s cluster manages nodes which in the context of AWS are EC2 instances i.e. virtual machines. K8s also conceptualized a logical layer called pods on top of nodes. Each pod also has its own allocated resources. While recent trends in auto-scaling research focus on the infrastructure layer of the cloud, such as Virtual Machines (VMs), similar strategies can be applied at the pod level too.

II. MOTIVATION

Data centers consume a significant amount of energy produced globally, with the advent of AI and High-Performance Computing it is set to increase further. Data center power consumption is projected to grow significantly over the next decade. Estimates suggest that global data center power demand will increase by approximately 160% by 2030 [4].

Digital transformation incorporates digital technologies in every aspect of our lives and, hence, demands an unparalleled increase in cloud computing to enable scalability, flexibility, and on-demand services. With organizations moving to support cloud-based solutions such as data analytics, artificial intelligence, IoT, and remote work, there is going to be an unparalleled surge in demand for the cloud. The higher demand calls for better resource optimization to avoid over-provisioning or under-utilization that may have more costs and energy consumption.

With increasing workloads, better optimization of resources will be needed to make cloud infrastructure support performance and cost-efficiency. Auto-scaling, predictive resource allocation, and energy-efficient computing are some of the techniques required for handling variable

workloads, reducing operational expenditures, and supporting sustainability objectives. This research work is a step towards achieving the above-mentioned goals.

The proposed approach ensures that the resources of a node in Kubernetes environment are used efficiently, allowing it to host more pods without the need to add additional nodes to the cluster. The proposed architecture provides a credible approach for estimating the required resources to run an application without compromising SLAs and QoS, thereby contributing to the efforts of green computing.

III. RELATED WORK

Kubernetes, while powerful, lacks a robust dynamic resource-provisioning mechanism. It forces users to manually implement resource-provisioning strategies and lacks flexibility, as it does not allow the integration of custom algorithms. Chang C et al. developed a generic platform for dynamic resource provisioning in Kubernetes that implements Monitor-Analyze-Plan-Execute (MAPE) loop for resource provisioning [5]. This approach improves resource efficiency but requires the users to craft algorithms that work for their service.

David Balla et al. proposed adaptive scaling of Kubernetes pods named Libra, which handles both horizontal and vertical scaling [6]. The main idea here is to overcome the limitations of traditional autoscalers. Horizontal Pod Autoscaling (HPA) triggers scaling events based on CPU or memory utilization but uses thresholds defined by human operators. Vertical Pod Autoscaling (VPA) adjusts resource requests based on historical data, but this can lead to inefficient resource usage if the environment or load changes rapidly. Rossi proposed a reinforcement learning model for both horizontal and vertical autoscaling, designed to guarantee that applications meet their required response times [7].

Advancements in artificial intelligence led to Long Short-Term Memory networks (LSTMs), an architecture designed for sequential model prediction. LSTMs were also utilized by G, B., & Jayalakshmi, S to forecast resource allocation [8]. Siqiao Xue et al. also discussed the use of Informer transformers and Reinforcement Learning for predictive auto scaling [9]. Since the historical resource usage of an application can be treated as a time series application. Further, Singh and Chana discussed emerging trends such as integrating machine learning models to predict workload patterns and improve energy efficiency in provisioning [10]. Raugust et al. conducted experiments using GPU tailored algorithms to increase the Virtual Infrastructure(VI) survivability to ensure request reliability of a service without increasing the cost [11]. Table I summarizes the techniques, metrics, scaling types, and SLAs of the work done in this field.

TABLE I. SUMMARY OF RELATED WORK

Ref	Technique	Input Metrics	Scaling type	SLA
[5]	MAPE loop	CPU & Memory	Horizontal	Application QoS
[6]	Mixed scaling algorithm	CPU	Vertical & Horizontal	-
[7]	Reinforcement learning	CPU	Vertical & Horizontal	Response time
[8]	LSTM	CPU & Memory	Vertical	Application QoS
[9]	Meta reinforcement learning	CPU	Horizontal	Application QoS

IV. ARCHITECTURE

The architecture presented for resource optimization in Kubernetes environments leverages a combination of monitoring, AI-driven decision-making, and resource orchestration to enhance efficiency in cloud infrastructure management as shown in Fig. 2. The system comprises four key layers, Resource Monitoring, Data Processing and Model Training, Resource Optimizer, and Resource Orchestration. As Singh K. discussed about the different approaches to develop and deploy the models, this paper utilizes AWS SageMaker approach [12]. Details of each layer is listed below:

A. Resource Monitoring Layer

This layer is responsible for collecting real-time metrics regarding resource usage from Kubernetes pods. Each pod contains two primary components:

1) *Application Container*: The main application or service running inside the Kubernetes pod.

2) *Resource Usage Agent*: A sidecar container within the pod that gathers fine-grained resource usage metrics such as CPU, memory, and disk I/O. The Resource Usage Agent provides deeper insights into the pod's resource consumption.

These metrics are collected by Prometheus, an open-source monitoring and alerting toolkit [13]. It scrapes metrics from the Resource Usage Agent and aggregates them for further processing. This setup provides both real-time and historical data for detailed analysis, enabling proactive resource management.

B. Data Processing and Model Training Layer

The layer focuses on processing the collected metrics and building machine learning models to optimize resource allocation. The components in this layer include:

1) *S3 Bucket*: Prometheus forwards the collected metrics to an Amazon S3 Bucket [14], where the data is stored for future analysis and training purposes using a procedure called Remote write. This storage acts as a central repository for both real-time and historical data, enabling long-term trend analysis.

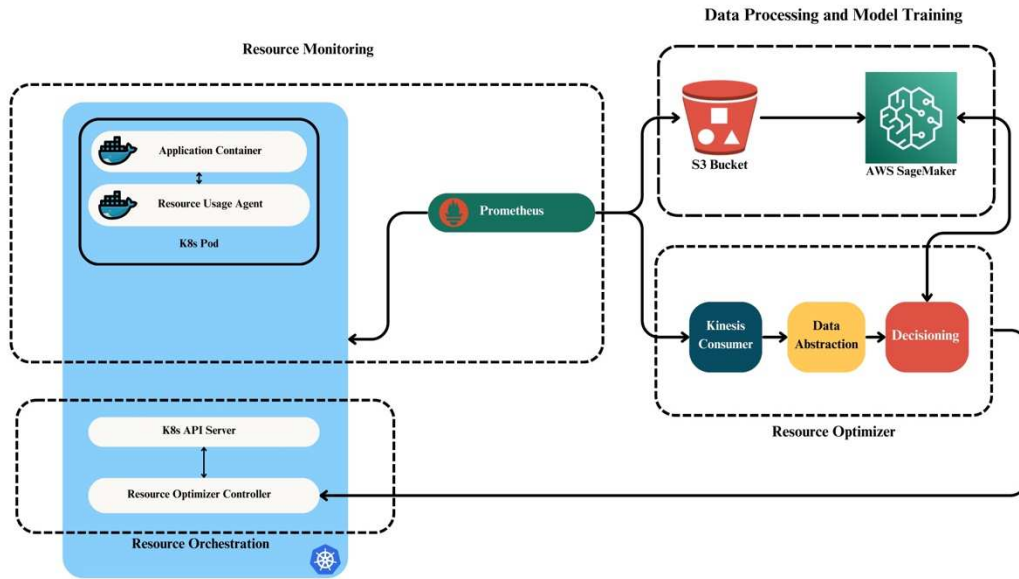


Fig. 2. Efficient Resource Management of Kubernetes Pod Architecture

1) *AWS SageMaker*: The resource usage data stored in the S3 bucket is fed into AWS SageMaker, which trains machine learning models using the data. These models are designed to predict future resource needs, allowing the system to make proactive scaling decisions. SageMaker automates the entire machine learning workflow, including data preprocessing, model training, and deployment. Once the models are trained, they are deployed and used for real-time inference to provide optimal resource allocation decisions. This allows the system to dynamically adjust resources based on predictive insights, ensuring that pods are neither over-provisioned nor under-provisioned.

C. Resource Optimizer Layer

This layer processes real-time metrics integrates with machine learning models and determines the necessary scaling or resource adjustments. The components involved include:

1) *Kinesis Consumer*: Real-time data streams from Prometheus are ingested into Amazon Kinesis, a real-time data streaming service. The Kinesis Consumer retrieves the data streams and prepares them for further abstraction and analysis.

2) *Data Abstraction*: This component processes and structures the raw data from Kinesis, making it usable by the decision-making models. It prepares the data for

feeding into the prediction algorithms trained in AWS SageMaker.

3) *Decisioning*: The processed data is then passed to the Decisioning module, which applies the machine learning models trained by SageMaker to make real-time decisions about resource allocation. This module decides adjusting the resource limits of specific pods based on workload predictions.

D. Resource Orchestration Layer

The Resource Orchestration Layer is responsible for implementing the decisions made by the optimizer. This layer interacts directly with the Kubernetes infrastructure to enact scaling and resource adjustments:

1) *Resource Optimizer Controller*: The Resource Optimizer Controller is a custom Kubernetes controller that executes the actions based on the decisions provided by the Resource Optimizer Layer. It interacts with the Kubernetes API to dynamically adjust resources for each pod based on real-time and predicted requirements.

2) *Kubernetes API Server*: The orchestration layer communicates with the Kubernetes API Server to modify resource configurations, such as scaling the number of pods or adjusting CPU and memory limits for running pods.

V. EXECUTION FLOW

The execution flow of the proposed resource optimization system operates in a continuous cycle of monitoring, data abstraction, and real-time resource adjustment within a Kubernetes environment as shown in Fig. 3. This section outlines the detailed flow of execution.

The process begins within the initiation of resource monitoring within Kubernetes pods. As mentioned in the above architecture, resource usage agent collects real-time metrics on resource usage and further prometheus scrapes the data from it at regular intervals usually near real time. The Resource Optimizer consumes the events over a predefined time window typically set to couple of hours and processes them i.e. consolidates them into a single abstracted representation of the resource usage of a pod during the decision window. Finally, it is encoded then fed to the decisioning module of the Resource Optimizer.

The decisioning module assesses whether the resource usage patterns indicate inefficiency or over-provisioning. If the system deems that resources are already optimized, the flow returns to continuous monitoring. Otherwise, the system engages the Resource Orchestration Layer. This layer communicates with the Kubernetes API Server to execute necessary changes, such as scaling pods, adjusting CPU and memory limits, or rebalancing workloads across nodes. The orchestration ensures that the resources are aligned with the predicted and current workload demands, avoiding under or over-provisioning.

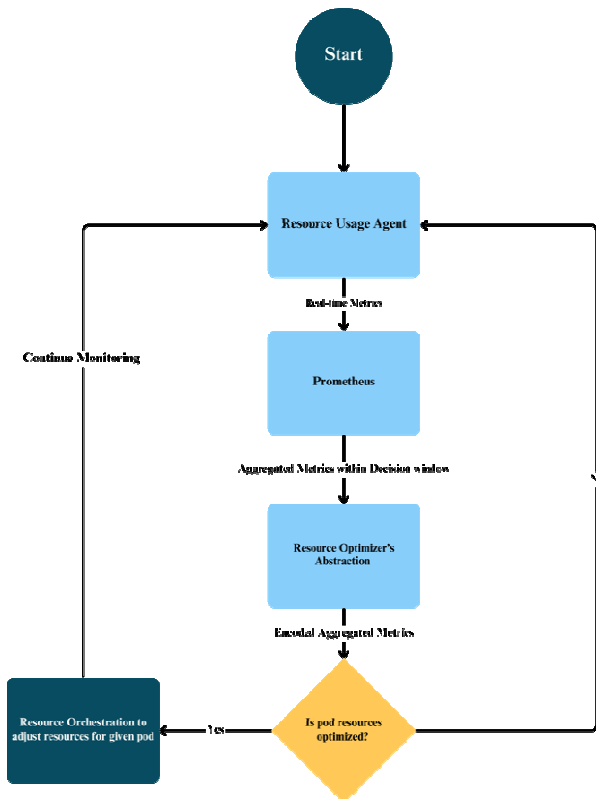


Fig. 3. Execution flow of Resource Optimizer

After adjusting the resources, the system loops back to the Resource Usage Agent, continuing to monitor the newly allocated resources. This forms a continuous cycle, where resource utilization is regularly analyzed, and adjustments are made in real time to maintain optimal resource efficiency.

VI. ADVANTAGES OF THE PROPOSED ARCHITECTURE

The following are the advantages of the architecture

1) *Efficient Resource Utilization*: By monitoring pod-level resource consumption in real-time using the Resource Usage Agent and adjusting resource allocation accordingly, the architecture ensures that no resources are wasted. Pods are given just the right amount of CPU and memory, reducing over-provisioning and under-utilization, which leads to more efficient use of computational resources.

2) *Seamless Integration with Kubernetes*: The architecture leverages Kubernetes' native capabilities, such as the Kubernetes API server and custom controllers. This ensures that it can easily integrate into any Kubernetes-based environment without significant modifications.

3) *Cost Efficiency*: By optimizing resource allocation and minimizing over-provisioning, this architecture can lead to significant cost savings, particularly in cloud environments where resources are billed based on usage. The predictive models help ensure that resources are only allocated when necessary, reducing unnecessary cloud spend.

4) *Modular and Extensible*: The architecture is modular, with clear separations between the monitoring, data processing, and orchestration layers. Each layer can be independently improved or scaled as needed. For example, different AI models can be experimented with in SageMaker without needing to modify the monitoring layer or orchestration logic.

5) *Energy Efficiency and Sustainability*: Through intelligent resource optimization, this architecture helps minimize the use of unnecessary resources, which contributes to the overall goal of Green Computing. By reducing resource consumption during low-demand periods and optimizing energy use, it aids in reducing the carbon footprint of cloud infrastructure.

6) *Low Latency*: With the help of continuous monitoring, resources can be examined in near real time. Further Resource Optimizer and Resource Orchestration layers applies the updates to the underlying resources of the pods in near real time. Thus, it provides low latency.

VII. RELATED EXPERIMENTS

G, B., & Jayalakshmi has conducted experiments using LSTM approach and applied the predicted resources to scale the resources to the nodes of the Kubernetes Cluster. The Dataset used in this experiment is the traces are gathered during a three-month period in the dispersed Materna Data Centers in Dortmund. The three datasets consist of the running 527 VMs in trace 1, 527 VM's in trace 2 and 547 VM's in trace 3 respectively of 69 cores and 6780 GB RAM. The workloads in the tracked VMs are mission critical [8]. In this experiment, the adaptive moment estimation (ADAM) was used as optimizer and Vanilla Long Short-Term Memory Sequential Model (LSTM) was used as the architecture.

Table I shows the prediction samples made by the model [8], the model predicts higher CPU as well as Memory usage than actual CPU and Memory Usage.

TABLE II. PREDICTIVE AUTO-SCALING USING LSTM METHOD

Date Time	CPU Usage in MHz (Actual)	Memory Usage in KBs (Actual)	CPU Usage in MHz (Actual)	Memory Usage in KBs Predicted
02/02/16 00:00:00	58	115763	81.240875	136286.890625
02/02/16 00:05:00	123	332189	106.497734	405408.000000
02/02/16 00:15:00	49	198810	66.726730	151165.390625

However, the G, B., & Jayalakshmi, S. calculated the error rate using RMSE and it came out to be 0.0081 which is well within the range of acceptable values. Fig. 4. and Fig. 5. depicts the plots of memory and CPU resources predicted by LSTMs [8].

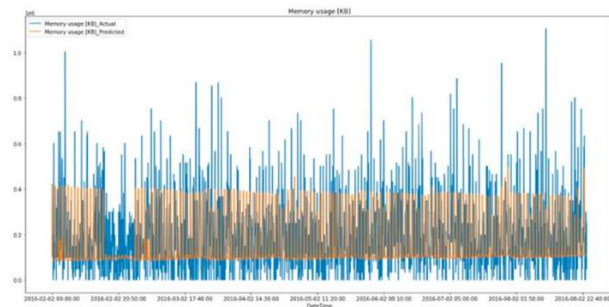


Fig. 4. Plot of Memory Usage in Actual and Predicted using LSTMs

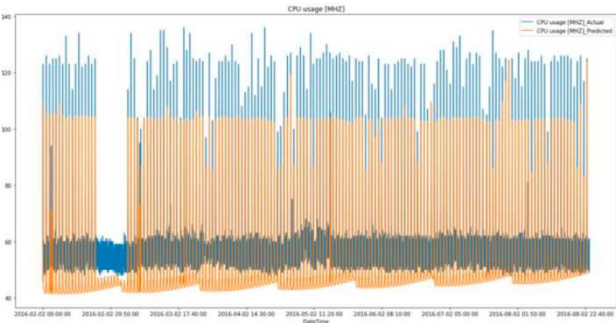


Fig. 5. Plot of CPU Usage in Actual and Predicted using LSTMs

VIII. CONCLUSION AND FUTURE WORK

This paper presents a comprehensive approach to addressing the challenge of efficient resource management at the Kubernetes pod level using Artificial Intelligence. By leveraging modern machine learning techniques, such as transformers, for resource prediction, the proposed architecture provides a proactive and adaptive solution to dynamically allocate resources within cloud environments. The integration of real-time monitoring, AI-driven decision-making, and seamless orchestration through Kubernetes allows the system to optimize resource allocation, ensuring that each pod receives the precise number of resources it requires, while avoiding over-provisioning or under-utilization.

Further research can be conducted to enhance the presented architecture and broaden its application. Investigating more sophisticated AI models, such as using transformers could lead to more accurate predictions of resource requirements for various workloads. Additionally, exploring reinforcement learning methods like Proximal Policy Optimization (PPO) may offer improved decision-making in dynamic cloud environments.

Incorporating a broader range of resource metrics, such as network bandwidth, disk I/O, and latency, would allow for more granular optimization of cloud infrastructure. This could lead to more efficient handling of complex, latency-sensitive workloads. Also, conducting large-scale tests in real-world cloud environments would provide valuable insights into the performance and scalability of the system. Simulating real-world usage patterns could validate the architecture's effectiveness and identify areas for further improvement.

Finally, developing specific energy-efficiency metrics and tracking the reduction in energy consumption due to optimized resource allocation would provide a quantifiable measure of the system's contribution to green computing. This could help organizations justify the adoption of AI-driven resource management from both an operational and environmental perspective.

REFERENCES

- [1] T. Lorigo-Botran, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," *Journal of Grid Computing*, vol. 12, no. 4, pp. 559-592, 2014.
- [2] G. Kaur, A. Bala, and I. Chana, "An intelligent regressive ensemble approach for predicting resource usage in cloud computing," *Journal of Parallel and Distributed Computing*, vol. 123, pp. 1-12, 2019.
- [3] Kubernetes Documentation, "Kubernetes overview," *Kubernetes.io*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/overview/>. [Accessed: Nov. 3, 2024]
- [4] Goldman Sachs, "AI is poised to drive 160% increase in data center power demand," *Goldman Sachs*, May 14, 2024. [Online]. Available: <https://www.goldmansachs.com/insights/articles/AI-poised-to-drive-160-increase-in-power-demand>. [Accessed: Nov. 3, 2024]

- [5] C.-C. Chang, S.-R. Yang, E.-H. Yeh, P. Lin, and J.-Y. Jeng, "A Kubernetes-based monitoring platform for dynamic cloud resource provisioning," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, 2017. doi:10.1109/GLOCOM.2017.8254046
- [6] D. Balla, C. Simon, and M. Maliosz, "Adaptive scaling of Kubernetes pods," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020, pp. 1–5
- [7] F. Rossi, "Auto-scaling policies to adapt the application deployment in Kubernetes," in *Proc. CEUR Workshop*, 2020, vol. 2575, pp. 30–38
- [8] G. B., & Jayalakshmi, S. (2021). Predictive Scaling for Elastic Compute Resources on Public Cloud Utilizing Deep Learning based Long Short-term Memory. *International Journal of Advanced Computer Science and Applications*, 12(10). <https://doi.org/10.14569/ijacsa.2021.0121010>
- [9] Xue, S., Qu, C., Shi, X., Liao, C., Zhu, S., Tan, X., Ma, L., Wang, S., Wang, S., Hu, Y., Lei, L., Zheng, Y., Li, J., & Zhang, J. (2022). A meta reinforcement learning approach for predictive autoscaling in the cloud. *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. <https://doi.org/10.1145/3534678.3539063>
- [10] S. Singh and I. Chana, "Cloud resource provisioning: Survey, status and future research directions," *Knowledge and Information Systems*, vol. 49, pp. 1005–1069, 2016
- [11] A. S. Raugust, W. J. Loch, F. R. D. Souza, M. A. Pillon, C. C. Miers, and G. P. Koslovski, "Towards virtual infrastructure allocation on multiple IaaS providers with survivability and reliability requirements," *International Journal of Grid and Utility Computing*, vol. 10, no. 6, p. 708, 2019. doi:10.1504/IJGUC.2019.102752
- [12] K. Singh, "From development to deployment: Streamlining MLOps with monoliths, microservices, and Amazon SageMaker," *International Research Journal of Engineering and Technology*, vol. 11, no. 9, p. 16, 2024. [Online]. Available: <https://www.irjet.net/archives/V11/i9/IRJET-V111902.pdf>.
- [13] Prometheus, "Prometheus," GitHub repository, 2024. [Online]. Available: <https://github.com/prometheus/prometheus>. [Accessed: Nov. 3, 2024]
- [14] Amazon Web Services, "Amazon Simple Storage Service (S3)," AWS, 2024. [Online]. Available: <https://aws.amazon.com/s3/>. [Accessed: Nov. 3, 2024]
- [15] Amazon Web Services, "Amazon SageMaker – Machine Learning," AWS, 2024. [Online]. Available: <https://aws.amazon.com/sagemaker/>. [Accessed: Nov. 3, 2024]
- [16] Amazon Web Services, "Amazon Kinesis – Real-Time Data Streaming," AWS, 2024. [Online]. Available: <https://aws.amazon.com/kinesis/>. [Accessed: Nov. 3, 2024]